Demonstrating the live placement controller

A step-by-step guide to showing the testbed, the dashboard and the result to someone who has never seen them

Latency-Aware Edge Game Server Placement ▪ Ankam Charan Teja, Aranya Roy, Nikhil Kumar Reddy, Yashvardhan Deep, Akshit Singh
https://github.com/ankamteja/latency-aware-edge-game-placement

What you are showing, in one breath

Edge placement algorithms choose a node with a latency formula, *propagation delay + size / bandwidth*. It has a term for distance and none for load. On a real, small edge node, load is what decides latency: the nearest node is the fastest until it is crowded, then it is the slowest by seconds. The demo shows a real game-server testbed, with real bots, where a controller using that formula leaves players on an overloaded node, and a controller using a load-aware model learned from measurements moves them off within seconds. The dashboard only displays; everything is driven from a terminal, so the audience can see that nothing is scripted behind the page.

> **Say** "Every edge-placement paper and simulator ranks nodes by distance. We built a real testbed and measured what that misses. Watch what happens to the nearest node when players pile onto it, under the standard formula and under ours."

1 Before the audience arrives (10 minutes)

1. **Plug in the charger and set the power mode to Performance** (GNOME: top-right menu, Power Mode). This matters: in Balanced mode every node is two to three times slower than the models were fitted on (results chapter, section 8.6). Unplugging the charger silently switches it back.
2. **Close heavy apps** (other browsers, video, builds). The four game servers, up to 160 bots and the dashboard share one laptop.
3. **Bring the testbed up and check it.** After a reboot the container links are gone and must be recreated; otherwise only the check is needed:

```
cd ~/latency-aware-edge-game-placement
topology/up.sh            # only after a reboot; ends with ALL CHECKS PASSED
topology/verify.sh        # must end with ALL CHECKS PASSED
```

4. **Start the dashboard** in its own terminal and leave it running:

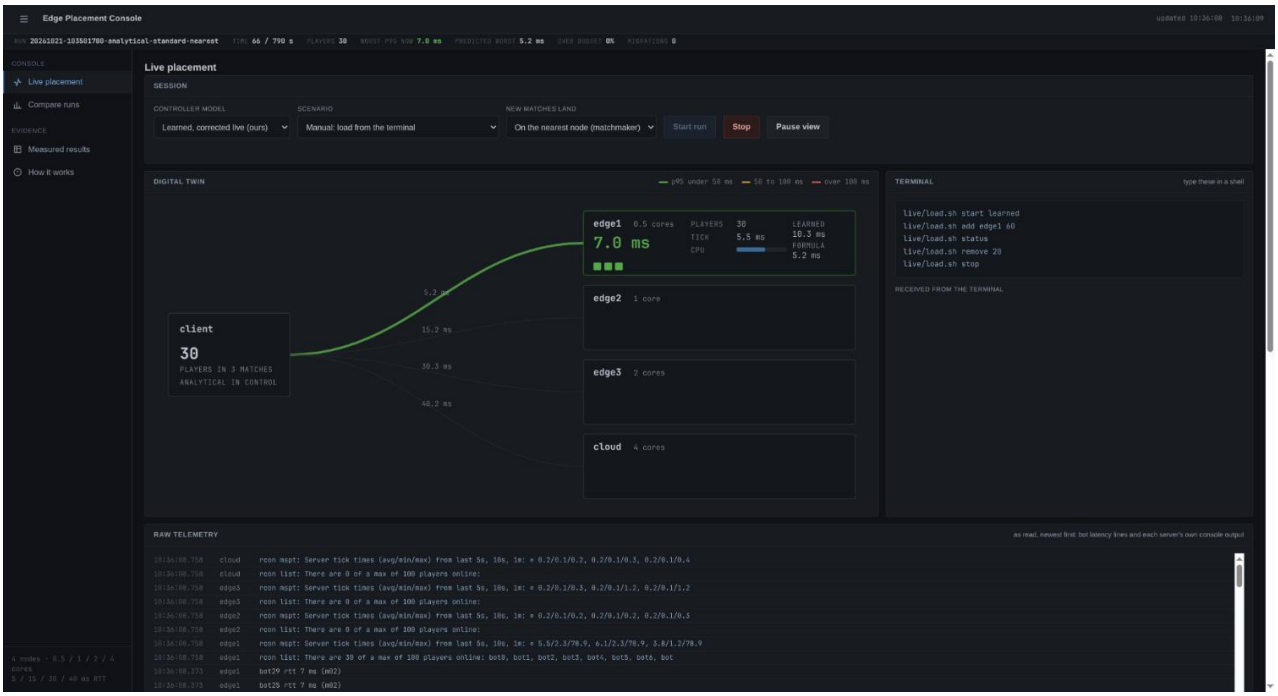
```
live/run.sh               # prints: dashboard: http://127.0.0.1:8090
```

5. Open `http://127.0.0.1:8090` in a browser, press `Ctrl+Shift+R` once (loads the newest page), and make the window full screen.
6. Open a second terminal next to the browser, large font, in the repository folder. This is the one the audience watches you type into.

Check before you start: the top strip of the dashboard shows no run, **Start run** is enabled, and `topology/verify.sh` passed in the last few minutes. If a previous session was left running, `live/load.sh stop` ends it and disconnects every bot.

No testbed available (another laptop, a projector PC, a viva online)? Open `https://ankamteja.github.io/latency-aware-edge-game-placement/`. It is the same dashboard playing back the runs recorded on 2026-10-21 from their logs: pick a controller and a scenario, press **Start run**, choose the speed (4x keeps a 13-minute run under 4 minutes). Demos 1 to 3 below become "analytical, standard", "learned, standard" and "learned, corrected live, capacity drop". Say that it is a replay; nothing in it is simulated.

2 A tour of the screen (2 minutes)



Top strip
the run's name, elapsed time, players connected, the *worst* node's measured p95 right now, what the controller *predicted* for it, the share of time over the 100 ms budget, and how many matches have been moved. The update time on the right turns amber if the feed stops.

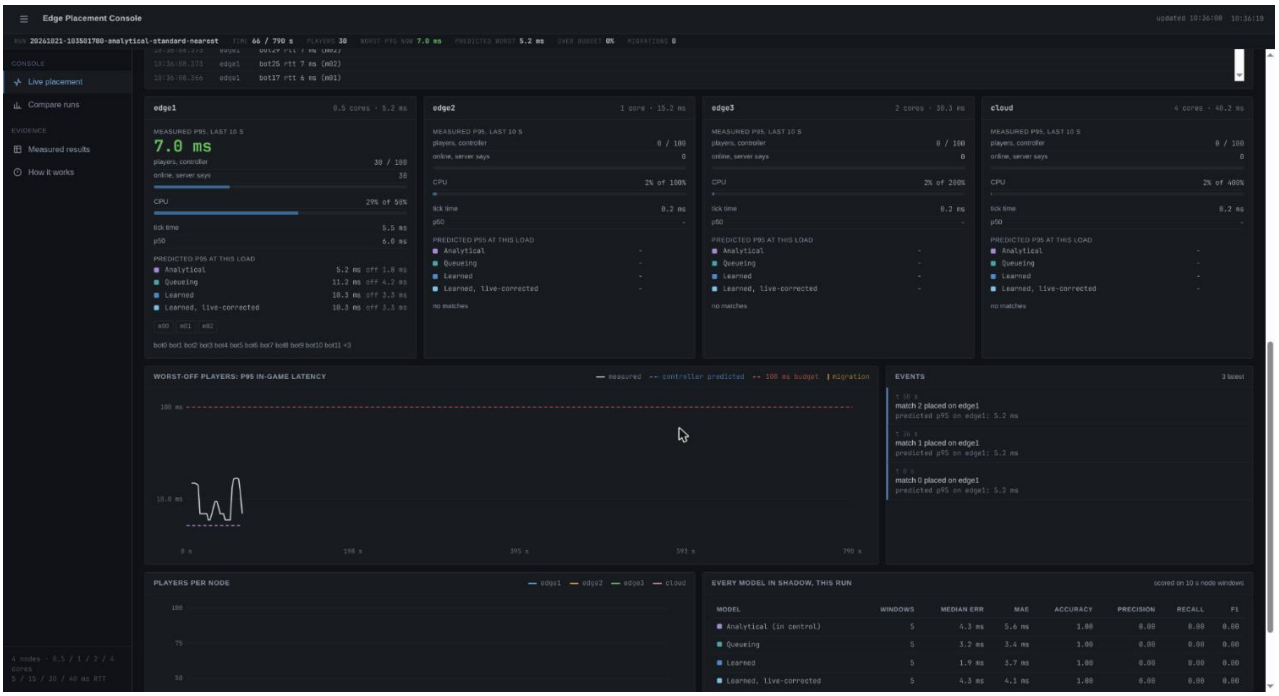
Session
which model controls placement, the scenario, and where new matches land. While a run is going the form shows that run and is locked.

Digital twin
the client on the left, the four servers on the right, each line labelled with its measured network delay (5, 15, 30, 40 ms). Each server shows players, server tick time, CPU, the measured p95 in large type (green under 50 ms, amber 50 to 100, red above 100), what the learned model and the formula predict at this load, and one square per match of ten players. A moving dot is a match being migrated.

Terminal
the commands you can type, and every command received from a terminal, as it arrives.

Raw telemetry
the data itself, newest first: one line per bot latency sample (`bot23 rtt 7 ms (m02)`) and each server's

own console answers (`rcon list: There are 40 of a max of 100 players online:` followed by the bot names). Point at this when someone asks whether the numbers are real.



Further down: one card per node with the measured p50/p95, the players the controller placed next to the players the server itself reports, CPU against the node's quota, and every model's prediction for the node at this load with how far off it is; the worst-player latency over time with the controller's prediction and the migrations marked; the event log; players per node over time; and a table scoring every model in the background on this run's own measurements (median error, F1 at 50 and 100 ms).

3 The fastest way: the guided script

```
live/demo.sh            # our model, then the formula, about 12 minutes
live/demo.sh ours       # only our model
live/demo.sh formula    # only the formula
```

Each step prints the command, waits for `Enter`, runs it, and shows the live table for a minute with one line on what to point at. Keep the dashboard visible next to the terminal. The sections below are the same steps typed by hand, with what to say.

4 Demo 1: the formula puts everyone on the nearest node (3 minutes)

```
live/load.sh start analytical     # a session with the formula in control
live/load.sh join 70              # 70 players arrive, one match every 5 s
live/load.sh status               # the twin, as a table
```

What happens: each arriving match is placed by the formula. It predicts about 5 ms for edge1 at any load, so every match goes to edge1. The event log shows each decision with every node's prediction ("placed on edge1 by the analytical model, predicted 5 ms; other nodes: edge2 15, edge3 30, cloud 40 ms"). Within a minute edge1's p95 is in the seconds and its card is red, and the formula never moves

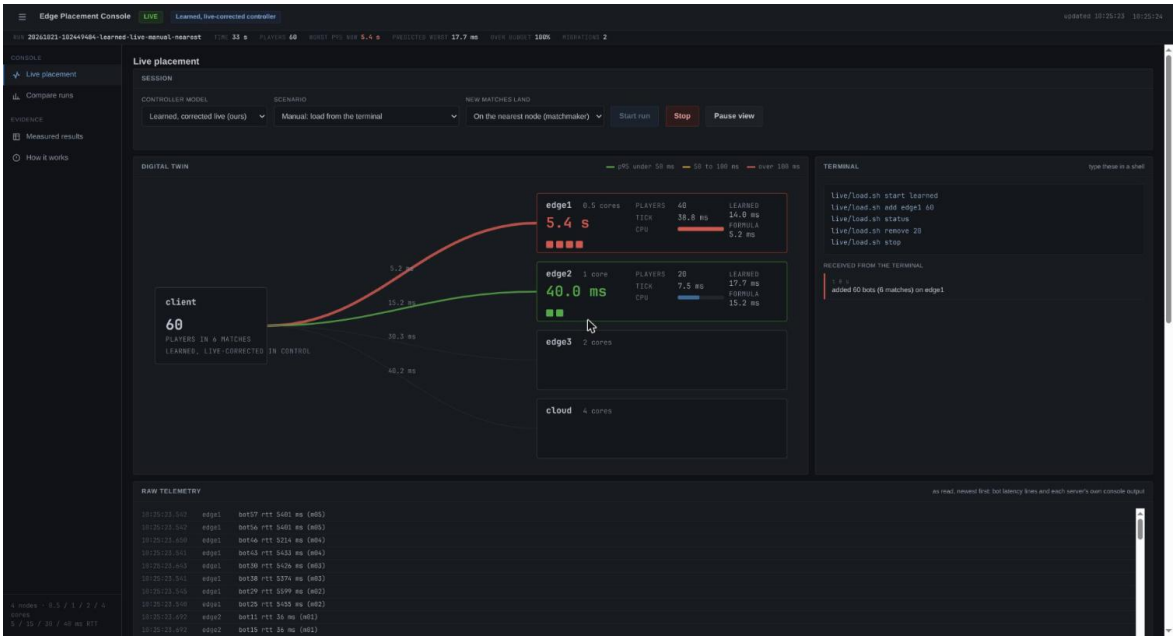
anyone. Tested on 2026-10-21: **all 70 players on edge1, predicted 5 ms, measured 12.8 s**.

> **Say** "This is the standard model. It predicts 5 milliseconds because edge1 is 5 milliseconds away. The players on it are waiting over 12 seconds, and it will never move them: from its point of view edge1 is still the best node."

```
live/load.sh stop
```

5 Demo 2: our model places players and keeps them moving (5 minutes)

```
live/load.sh start learned-live   # our model, corrected from live measurements
live/load.sh join 40              # players arrive; the model picks each node
live/load.sh join 30              # more players
live/load.sh add edge1 30         # a flash crowd forced onto edge1
```



What happens: for each arriving match the model predicts every node's p95 and picks the lowest worst case. The first matches go to edge1, the closest node, while it is predicted fast (7, 8, 10, 14 ms). Once edge1 is predicted past its knee, new matches go to edge2 and edge3. Then `add edge1 30` forces three matches onto edge1, the way a distance-only matchmaker would; every 10 s the controller moves one off, shown as a dot travelling along the links. Tested on 2026-10-21: four matches to edge1, the next three to edge2, and the forced crowd moved to edge3 within 36 s, leaving every node at 11 to 34 ms. 40 players is edge1's knee: while 40 players log in at once, edge1 can briefly tip over, and the live model then raises its estimate for edge1 and moves a match off. Point at that if it happens.

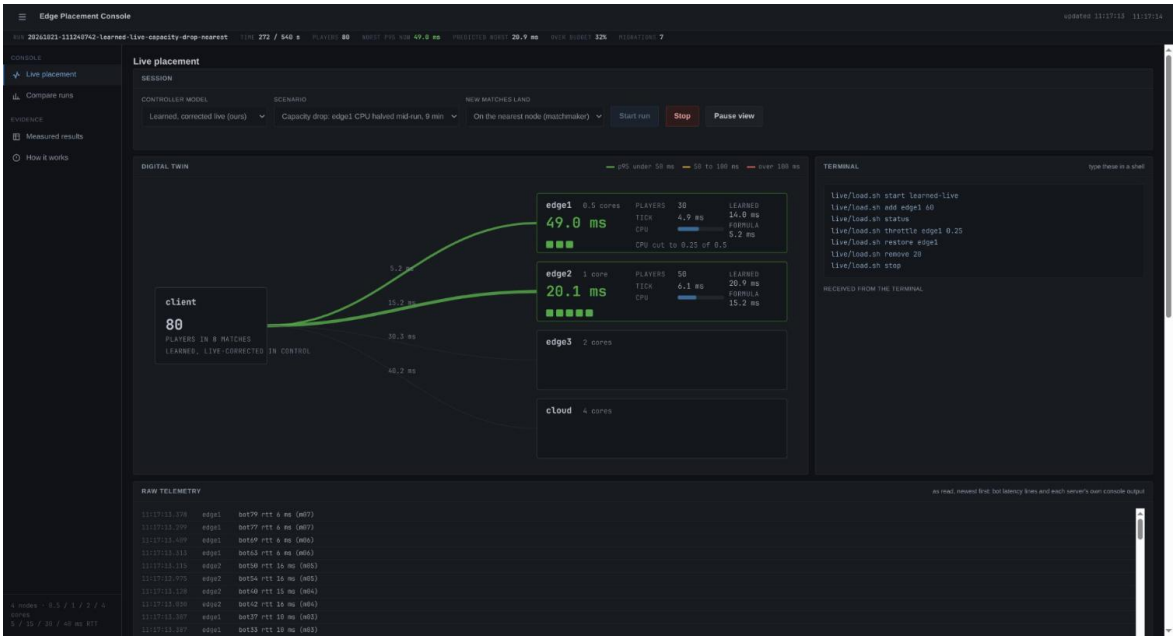
> **Say** "Same players, same servers. This model learned from measurements that a crowded edge is slow, so it places the next players elsewhere before the edge breaks. When a crowd is forced onto it anyway, it moves one match at a time; you can see each move and its predicted gain in the event log."

6 Demo 3: a node gets slower and nobody is told (3 minutes)

Leave Demo 2 running, then halve edge1's CPU from the terminal. No model is told.

```
live/load.sh throttle edge1 0.25  # edge1 now has a quarter core instead of half
live/load.sh status
```

What happens: edge1's p95 jumps; the learned model's offline curve still predicts a healthy node, but the live-corrected model compares each 10 s window with its prediction and estimates how much more each player now costs edge1. Tested on 2026-10-21: after the cut it moved a match off edge1 within 25 s, and edge1 held 20 players at 34 ms. The twin then shows the cut on edge1's card in red:



> **Say** "A noisy neighbour, a throttled host, a smaller VM: the node got slower and nothing told the controller. It noticed from the measurements alone and estimated a factor of two, which is exactly what we did to it."

```
live/load.sh restore edge1        # give edge1 its CPU back
live/load.sh remove 20            # take two matches off
live/load.sh stop                 # end the session, disconnect every bot
```

7 The recorded comparisons (2 minutes)

Open **Compare runs**. Every finished run is listed with its controller, worst-player p95 (median and maximum), time over budget, samples over budget, migrations and the mean reconnect time; tick two runs to overlay their worst-player latency. The table below is the same comparison measured on a fixed 13-minute schedule (up to 140 players arriving and leaving the same way for both), and a 9-minute run in which edge1's CPU is halved at 220 s.

Worst node's p95, 3 runs each, median [lowest, highest]	median	over 100 ms	moves
Standard, analytical controller	1.1 s [0.08, 4.5]	53% [46, 62]	0
Standard, queueing controller	99 ms [72, 166]	50% [46, 53]	11
Standard, learned controller	39 ms [33, 47]	33% [18, 41]	10
Standard, learned corrected live	33 ms [32, 34]	8% [5, 11]	15
CPU cut, queueing, after the cut		100% [99, 100]	4
CPU cut, learned (offline), after the cut		100% [99, 100]	4
CPU cut, learned corrected live, after the cut		6% [4, 13]	12 [8, 12]

Queueing loads edge2 to 80 players, past its knee, in every run; learned stops at 70 in every run. After the CPU cut, queueing and the offline learned model keep 40 players on the slowed edge1. The live-corrected model moves players off, but be precise if asked: it also learns from players logging in, so in two of its three runs it had already left edge1 with 0 and 10 players before the cut. Only the run with 30 players on edge1 at the cut shows it detecting the change (a match moved off 27 s later). The graphs of both comparisons are figures 6 and 7 in the repository's `figures/` folder.

> **Say** "This is the same schedule of players under each controller, recorded on the testbed, not simulated."

8 The offline evidence (2 minutes)

Measured results shows the grid behind the models: p95 against players for every node against what the formula predicts, each model's placement deployed for real at 40 to 160 players, and every model's out-of-sample score. The headline: with each model's placement deployed, the worst player's p95 was 26 / 29 897 / 17 758 / 19 990 ms for the formula at 40 / 80 / 120 / 160 players in the first round (16 / 28 244 / 18 658 / 28 251 ms in the second), and 16 to 55 ms for both load-aware models. **How it works** is the pipeline on one page: grid, dataset, models, and the live loop.

9 Questions you will get, and honest answers

Is this a simulation?
No. Five containers on one laptop: four real Minecraft (Paper 1.21.4) servers with real CPU limits and network delay set by Linux netem, and real protocol-level bots. The raw telemetry panel shows the samples and the servers' own console replies.

Does the learned model place better than queueing theory?
Not on the four offline scenarios: a correctly fitted M/M/1 model placed as well. The learned model predicts latency best (median error 0.9 ms, 1.1 ms on two held-out replicates, against 1.8 and 2.2 ms; on the held-out data the gap to queueing is within noise). The claim is that the load-blind formula fails, by orders of magnitude, and that any load-aware model avoids it.

Why is a migration a disconnect?
The controller stops the match on one server and starts it on another, so its ten players reconnect (about 6 s on average here). A production system would hand game state over first; our gap is an upper bound.

Why these core counts and delays?
They were chosen so that the trade-off exists. The claim is that an inversion exists, is large, and is avoided by a load-aware model; not that edge nodes break at a particular player count.

What would change on real hardware?
The knees move; their existence does not. Section 8.6 of the results lists every limit: one host, three replicates, synthetic players, and the power-mode effect itself, which moved edge1's knee threefold with nothing else changed.

10 If something goes wrong

You see	Do
Page does not load	`live/run.sh` is not running, or failed its check: run `topology/verify.sh` and read the failed line.
Start run disabled, "testbed busy"	A measurement script is running. Wait for it, or stop it.
Update time turns amber	The controller stopped sending. Look at the `live/run.sh` terminal for an error; restart it.
Players stay at 0 after add	Bots cannot join: `topology/verify.sh`; after a reboot, `topology/up.sh`.
Every node much slower than this guide	Power mode is not Performance, or the charger is unplugged. The run's event log warns about it at start.
A node still throttled after a crash	The next run restores it at start; `topology/verify.sh` reports it.

Cheat sheet

```
topology/verify.sh                 # ALL CHECKS PASSED
live/run.sh                        # dashboard on http://127.0.0.1:8090
live/demo.sh                       # the whole demo, one step per Enter
live/load.sh start analytical      # or learned, queueing, learned-live
live/load.sh join 40               # players arrive; the model places each match
live/load.sh add edge1 60          # bots forced onto one node, matches of 10
live/load.sh status
live/load.sh throttle edge1 0.25   # node gets slower, no model told
live/load.sh restore edge1
live/load.sh remove 20 [node]
live/load.sh stop
live/ab.sh standard                # formula vs learned, same schedule, ~30 min
```